

Sarvam Developer Portal Codebase Guide

Plain reference PDF explaining the project structure, routes, files, features, implementation decisions, and where to edit things. Generated on 17 May 2026.

Item	Details
Project	sarvam-intern-portal
Stack	Next.js 16, React 19, TypeScript, Tailwind CSS 4
Main routes	/playground, /diff-viewer, /documentation
Core features	Streaming inference playground, audio input, live metrics, model diff view, custom token-level diff algorithm

1. High-Level Overview

This app is a developer portal for the Sarvam frontend assignment. It lets a user test model inference in the browser, stream responses token by token, see live metrics, and compare two model outputs using a custom token-level diff view.

- The root URL redirects to /playground.
- The playground supports text input, audio input, streaming responses, stop streaming, markdown rendering, and live metrics.
- The diff viewer supports sample comparisons and live model comparisons.
- The app uses server API routes so the Sarvam API key is not exposed to the browser.
- The UI includes dark mode, sidebar navigation, toasts, responsive layouts, and accessibility basics.

2. Project Structure

File or Folder	Purpose
app/layout.tsx	Root shell. Wraps the app with ThemeProvider, ToastProvider, Sidebar, and main layout spacing.
app/page.tsx	Redirects the root page to /playground.
app/globals.css	Tailwind import, dark mode variant, base colors, and sr-only helper.
app/playground/page.tsx	Main inference playground UI and behavior.
app/diff-viewer/page.tsx	Model output comparison page.
app/documentation/page.tsx	In-app guide for how to use the portal.
app/hooks/useStream.ts	Client streaming hook using fetch and ReadableStream.
app/hooks/useAudioRecord.ts	Browser audio recording and transcription hook.
app/utls/diffAlgorithm.ts	Custom token-level LCS diff algorithm.
app/api/chat/route.ts	Server-side streaming chat proxy to Sarvam.
app/api/compare/route.ts	Server route for comparing two model outputs.
app/api/transcribe/route.ts	Server route for speech-to-text transcription.
componets/Sidebar.tsx	Desktop sidebar, mobile header, docs link, profile, theme toggle.
componets/ToastProvider.tsx	Custom toast notification system.
componets/ThemeProvider.tsx	Wrapper around next-themes.
README.md	Project overview and setup guide.

3. Application Shell

app/layout.tsx

This file controls the global layout. It imports global styles, sets metadata, loads the sidebar, and wraps the app in ThemeProvider and ToastProvider. The desktop layout uses left padding so route content does not sit under the fixed sidebar.

```
<ThemeProvider defaultTheme="dark" enableSystem={false}>
<ToastProvider>
<Sidebar />
<main className="lg:pl-[256px]">{children}</main>
</ToastProvider>
</ThemeProvider>
```

Why this approach: providers are mounted once at the root, so every route can use theme and toast state. The fixed sidebar remains stable while the main route changes.

app/page.tsx

This file redirects users to /playground immediately. The app opens on the actual working tool instead of a marketing or empty landing page.

app/globals.css

- Imports Tailwind CSS.
- Defines the custom dark variant.
- Sets root background and foreground colors.
- Defines .sr-only for screen-reader-only announcements.

4. Sidebar and Navigation

The sidebar is in componets/Sidebar.tsx. It provides app navigation and theme control. On desktop it is a fixed left sidebar. On mobile it becomes a top header with icon-only route links.

- Uses usePathname to highlight active routes.
- Uses useTheme from next-themes to toggle dark mode.
- Uses a hydration guard so the theme toggle does not mismatch between server and client.
- Has links for Inference Playground, Model Output Diff, and Documentation.
- Includes a dummy user profile and Sarvam-style gradient avatar.

5. Playground Page

The playground is implemented in app/playground/page.tsx. It is the main interface for testing inference.

Main capabilities

- Text input with Enter to send and Shift + Enter for a new line.
- Audio mode using browser microphone recording.
- Model selector for text and voice modes.
- Streaming assistant response rendered as chunks arrive.
- Stop streaming button that keeps partial output.

- Live token count, speed, and elapsed time metrics.
- Markdown rendering for assistant responses.
- Example prompt cards for quick demos.

Important local components

Component or Function	What it does
AssistantMarkdown	Renders common Markdown from model replies.
parseMarkdownBlocks	Parses headings, lists, paragraphs, blockquotes, code, and dividers.
renderInlineMarkdown	Handles inline code, bold, italic, and links.
ModelSelector	Dropdown for switching text and audio models.
PersonaAvatar	Animated Sarvam-style assistant avatar.
ThinkingPersona	Shows thinking dots and streamed response text.
CompactMetricsBar	Mobile-friendly metrics row.
RightSidebar	Desktop metrics panel.
sendPrompt	Creates user/assistant messages and starts streaming.
handleComposerAction	Sends when idle and stops when streaming.

Why the playground is built this way

- Streaming is separated into useStream so the UI code stays focused on state and layout.
- Audio recording is separated into useAudioRecord because MediaRecorder has its own lifecycle and edge cases.
- The assistant message is created before the network response arrives. This allows the UI to show a thinking state first, then fill the same bubble token by token.
- Metrics are computed while streaming rather than after completion, matching the assignment requirement.

6. Streaming Hook

`app/hooks/useStream.ts` owns the browser streaming logic. It posts to `/api/chat`, reads the response stream, parses SSE chunks, updates metrics, and supports aborting.

Part	Purpose
<code>startStream</code>	Starts the network request and stream reader.
<code>response.body.getReader()</code>	Reads chunks as they arrive.
<code>TextDecoder</code>	Converts stream bytes into text.
<code>parseSSEChunk</code>	Extracts <code>delta.content</code> from Sarvam/OpenAI-compatible SSE lines.
<code>requestAnimationFrame</code>	Batches UI updates for smoother streaming.
<code>metrics</code>	Stores token count and start time.
<code>AbortController</code>	Stops streaming and preserves partial response.

Why this approach: it directly uses Fetch API and ReadableStream, which the assignment explicitly asks for. The app does not wait for the full response before rendering.

7. Audio Recording Hook

`app/hooks/useAudioRecord.ts` owns microphone access, MediaRecorder setup, audio validation, transcription request, and audio errors.

- Requests microphone access with `navigator.mediaDevices.getUserMedia`.
- Uses MediaRecorder to collect audio chunks.
- Stores recorder and chunks in refs to avoid unnecessary rerenders.
- Rejects recordings shorter than 800ms or smaller than 900 bytes.
- Sends valid audio to `/api/transcribe`.
- Shows toasts for short recording, no speech, mic blocked, and transcription failure.

Why this approach: quick start/stop recordings often produce empty audio. The minimum duration and byte-size checks prevent confusing Sarvam errors and give the user a clear explanation.

8. API Routes

All Sarvam API access is handled server-side. The frontend calls local Next.js routes, and those routes call Sarvam with the server environment variable `SARVAM_API_KEY`.

`app/api/chat/route.ts`

- Uses Edge runtime.
- Accepts prompt or messages.
- Normalizes only valid chat messages.
- Calls Sarvam chat completions with `stream: true`.
- Returns the raw response body as `text/event-stream`.

`app/api/compare/route.ts`

- Accepts one prompt.
- Runs two Sarvam chat requests in parallel using `Promise.all`.

- Uses slightly different system prompts and temperatures for baseline and candidate outputs.
- Uses AbortController with a 15 second timeout.
- Returns modelA and modelB as JSON.

app/api/transcribe/route.ts

- Accepts FormData with an audio file.
- Forwards the audio to Sarvam speech-to-text using saaras:v3.
- Returns transcript JSON to the frontend.

Why server routes: they keep the API key secret, normalize input, and give the frontend simple endpoints.

9. Diff Viewer Page

The diff viewer is implemented in `app/diff-viewer/page.tsx`. It compares two model outputs for the same prompt. It supports sample data for reliable demos and live API comparison for real model outputs.

Main capabilities

- Sample data mode with predefined comparison examples.
- Use real model mode for API-backed comparison.
- Prompt box switches to live mode when the user types or clicks into it.
- Generate and compare button only appears in live mode.
- Baseline and Candidate response cards have Diff/Text toggles.
- Added tokens show in green; removed tokens show in red with strike-through.
- Summary metrics show total, added, removed, unchanged, match percentage, and algorithm info.

Important pieces

Piece	Purpose
mockScenarios	Local examples for deterministic diff demos.
ModeToggle	Switches Sample data and Use real model.
PromptCard	Prompt input, examples, Enter behavior, live-mode activation.
StatsCard	Radial match dial and metric cards.
AlgorithmMetric	Small card explaining token-level LCS.
ModelCard	Response card with avatar, copy button, Diff/Text toggle, scroll body.
HighlightedDiff	Renders added, removed, and unchanged tokens.
getDiffStats	Counts tokens and calculates percentages.

Why sample mode exists: it guarantees that the diff feature can be demonstrated even if the external API is slow, unavailable, or rate limited.

Why fixed card height exists: model outputs can be long. Fixed cards with internal scrolling keep the two response cards aligned for comparison.

10. Custom Diff Algorithm

The algorithm lives in `app/utlis/diffAlgorithm.ts`. It is custom-built because the assignment requires token-level diffing without external libraries.

How it works

- Tokenizes old and new text by splitting on whitespace while keeping whitespace tokens.
- Creates a dynamic programming matrix for Longest Common Subsequence.
- Fills the matrix by comparing each old token with each new token.
- Backtracks through the matrix to label tokens as unchanged, removed, or added.
- Returns an ordered array of DiffToken objects.

```
type DiffType = 'added' | 'removed' | 'unchanged'
interface DiffToken { type: DiffType; value: string }
```

```
Time complexity: O(n * m)
Space complexity: O(n * m)
```

Why LCS

- Line-level diffs are too coarse for model output comparison because a few words can change inside the same sentence.
- Token-level LCS clearly identifies shared words, added words, and removed words.
- The algorithm is easy to explain and deterministic.
- Myers diff is better for very large source-code diffs, but it is more complex. For short and medium model responses, LCS is a good fit.

Limitations

- It compares exact tokens, not semantic meaning.
- It is not optimized for very large documents.
- Punctuation can remain attached to nearby words depending on tokenization.
- Match percentage is based on unchanged tokens, not answer quality.

11. Documentation Page

app/documentation/page.tsx is an in-app guide. It explains how to use the playground and diff viewer, provides quick links, and gives a recommended demo path.

- Uses simple quick links to /playground and /diff-viewer.
- Lists steps for playground usage.
- Lists steps for diff viewer usage.
- Explains live metrics, audio flow, error states, and keyboard access.

12. Toast System

componets/ToastProvider.tsx implements a small toast system using React context. It avoids adding a UI dependency while still giving consistent notifications.

- toast() creates a toast and keeps the latest few notifications.
- dismiss() removes a toast.
- Toast variants include default, success, warning, and destructive.
- Destructive toasts use role=alert; others use role=status.
- Toasts auto-dismiss unless duration is Infinity.

13. Accessibility

- Chat conversation has aria-live=polite and aria-label=Chat conversation.
- Thinking and streaming states include screen-reader-only text.
- Icon buttons include aria-labels.
- Toast area uses aria-live and aria-relevant.
- Keyboard basics are supported: Enter sends, Shift + Enter makes a new line, buttons and links are focusable.
- Reduced motion is respected through prefers-reduced-motion CSS in playground animations.

14. Error Handling

Area	Issue	Handling
Stream	Network interruption	Partial output preserved and error toast shown.
Stream	User stops streaming	AbortController stops stream and shows stopped toast.
Prompt	Empty prompt	Send disabled or warning toast.
Audio	Short recording	Duration/size check blocks upload and shows warning.
Audio	Mic blocked	audioError and destructive toast.
Transcription	No transcript or API failure	User-facing message and toast.
Compare	Slow API	15 second timeout returns clean error.
Copy	Clipboard unavailable	Copy failed toast.

15. Where to Edit Things

Goal	File or area
Change playground examples	app/playground/page.tsx -> CHIPS
Change diff examples	app/diff-viewer/page.tsx -> mockScenarios
Change streaming behavior	app/hooks/useStream.ts and app/api/chat/route.ts
Change audio behavior	app/hooks/useAudioRecord.ts and app/api/transcribe/route.ts
Change compare timeout or prompts	app/api/compare/route.ts
Change diff algorithm	app/utls/diffAlgorithm.ts
Change diff response cards	app/diff-viewer/page.tsx -> ModelCard and HighlightedDiff
Change playground metrics	app/playground/page.tsx -> RightSidebar, CompactMetricsBar, BigMetric
Change diff metrics	app/diff-viewer/page.tsx -> StatsCard
Change toasts	componets/ToastProvider.tsx
Change sidebar navigation/profile	componets/Sidebar.tsx
Change docs page	app/documentation/page.tsx
Change dark mode default	app/layout.tsx -> ThemeProvider props

Rule of thumb: route-specific behavior lives in app/<route>/page.tsx. Shared app shell behavior lives in componets. Sarvam calls live in app/api. Reusable browser behavior lives in app/hooks. Algorithm behavior lives in app/utls.

16. Running and Deployment

Commands

```
npm install
npm run dev
npm run lint
npx tsc --noEmit
npm run build
```

Environment

```
SARVAM_API_KEY=your_sarvam_api_key_here
```

- Use Node.js 20.9 or newer.
- Local dev server is configured for http://127.0.0.1:3004.
- Set SARVAM_API_KEY in deployment environment variables.
- Verify /playground, /diff-viewer, and /documentation after deployment.

17. Demo Script

- Open /playground and send an example prompt.
- Show tokens, speed, and elapsed time updating while streaming.
- Stop a stream and explain that partial output is preserved.
- Switch to voice mode and explain audio transcription.
- Open /diff-viewer and click sample examples.
- Switch to live mode and generate a comparison.
- Show Diff/Text toggle and explain added/removed token highlights.

- Explain token-level LCS and $O(n * m)$ complexity.

18. Known Limitations

- Token count is approximate because it uses whitespace splitting.
- The diff algorithm compares exact token changes, not semantic equivalence.
- The LCS diff is not ideal for very large documents.
- The custom model dropdown can be improved with complete arrow-key menu navigation.
- Chat stream has manual stop and interruption handling; an explicit long-stall timeout could still be added.
- The file upload plus button is hidden because upload is out of scope.
- A future fleet deployment configuration page would complete the third workflow mentioned in the assignment situation.